Postfix SMTP Relay to Gmail using OAuth2

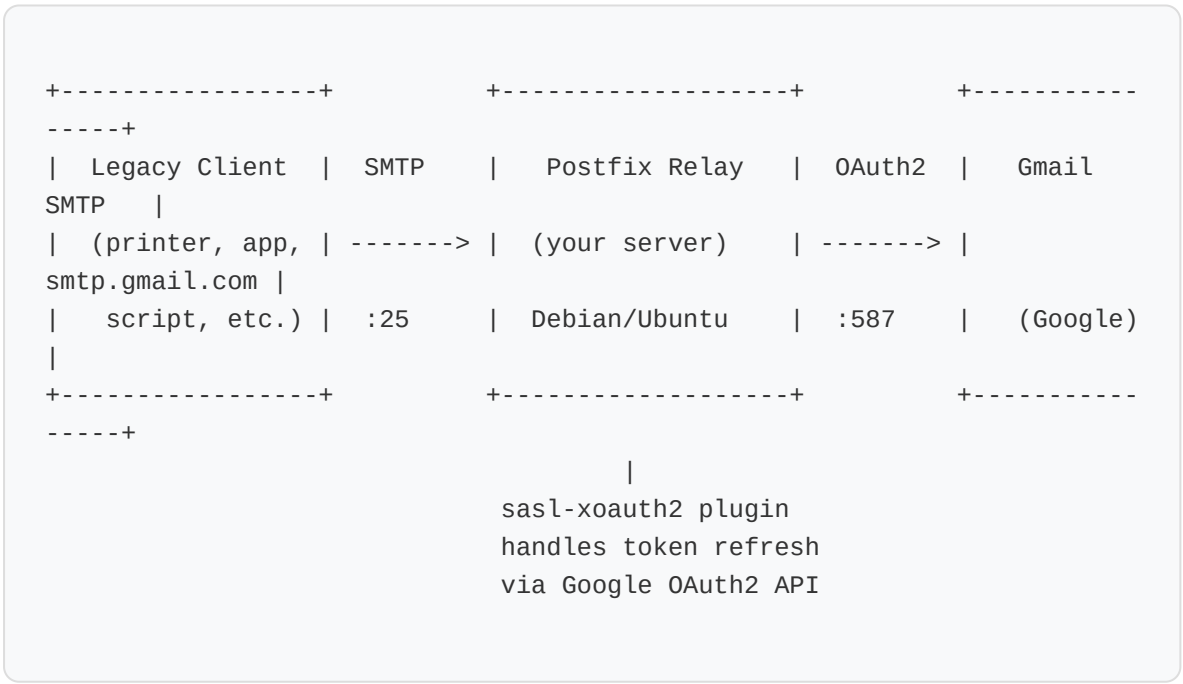
Configure Postfix on Debian/Ubuntu to relay mail through Gmail SMTP with modern OAuth2 authentication

Platform: Debian 12 / Ubuntu 22.04+ | Postfix 3.7+ | sasl-xoauth2 0.25+
Last updated: January 2026

1. Overview

Many legacy systems — printers, monitoring tools, ERP/CRM applications, scripts — need to send email but only support plain SMTP (username + password). Google is tightening security and **"Less Secure App" access has been permanently disabled**. App Passwords still work but require per-user 2FA and offer no token rotation.

This guide sets up **Postfix as a local SMTP relay** that accepts plain SMTP from legacy clients on your LAN and forwards mail to Gmail's SMTP servers using **OAuth2 (XOAUTH2)** authentication.



> **Note:** This guide works with both free Gmail accounts (`@gmail.com`) and Google Workspace accounts (`@yourdomain.com`). For Workspace, you may also consider configuring an SMTP relay in the Google Admin console for higher volume sending.

2. Prerequisites

Requirement	Details
Operating System	Debian 12 (Bookworm) or Ubuntu 22.04+ with root/sudo access
Gmail Account	A Gmail or Google Workspace account to send mail from (e.g., `postfix-relay@gmail.com`)
Google Cloud Project	Access to the Google Cloud Console to create OAuth2 credentials
Network	Outbound access to `smtp.gmail.com:587` and `oauth2.googleapis.com:443`
Build tools	`build-essential cmake libcurl4-openssl-dev libjsoncpp-dev libsasl2-dev`

3. Google Cloud Console Configuration

1 Create a Google Cloud Project

1. Go to https://console.cloud.google.com/
2. Click **Select a project → New Project**
3. Name it something descriptive (e.g., `postfix-smtp-relay`) and click **Create**
4. Select the newly created project

2 Enable the Gmail API

1. Navigate to **APIs & Services → Library**
2. Search for **Gmail API**
3. Click on it and press **Enable**

> **Note:** The Gmail API must be enabled even though we are using SMTP, because the OAuth2 scope `https://mail.google.com/` is part of the Gmail API.

3 Configure the OAuth Consent Screen

1. Go to **APIs & Services → OAuth consent screen**
2. Choose **Internal** (for Workspace) or **External** (for free Gmail accounts)
3. Fill in the required fields:
   - **App name:** `Postfix SMTP Relay`
   - **User support email:** your email address
   - **Developer contact:** your email address
4. Click **Save and Continue**
5. On the **Scopes** page, click **Add or Remove Scopes**
6. Add the scope: `https://mail.google.com/`
7. Click **Save and Continue** through the remaining steps

> **Important:** For **External** user type (free Gmail), the app will be in "Testing" mode. You must add the Gmail account you will use for sending as a **test user** under the OAuth consent screen settings. Testing mode apps support up to 100 test users and tokens are valid but need periodic re-consent.

4 Create OAuth2 Credentials

1. Go to **APIs & Services → Credentials**
2. Click **+ Create Credentials → OAuth client ID**
3. Application type: **Desktop app**
4. Name: `postfix-sasl-xoauth2`
5. Click **Create**
6. Note down the **Client ID** and **Client Secret**

> **Security:** Store the Client ID and Client Secret securely. They will be placed in a configuration file on your server with restricted permissions.

You should now have:

Value	Example
Client ID	`123456789-abcdefg.apps.googleusercontent.com`
Client Secret	`GOCSPX-xxxxxxxxxxxxxxxxxxxxxxx`
Gmail address	`postfix-relay@gmail.com`

4. Install Build Dependencies & sasl-xoauth2

1 Install build dependencies

```
sudo apt update
sudo apt install -y build-essential cmake pkg-config \
  libcurl4-openssl-dev libjsoncpp-dev libsasl2-dev
```

2 Build and install sasl-xoauth2

```
# Clone the sasl-xoauth2 repository
cd /tmp
git clone https://github.com/tarickb/sasl-xoauth2.git
cd sasl-xoauth2

# Build
mkdir build && cd build
cmake ..
make

# Install the SASL plugin
sudo make install
```

3 Link the plugin into the SASL directory

The build installs the plugin to `/usr/local/lib/sasl2/`, but the SASL library searches `/usr/lib/x86_64-linux-gnu/sasl2/`. Create a symlink so SASL can find the XOAUTH2 mechanism:

```
sudo ln -sf /usr/local/lib/sasl2/libsasl-xoauth2.so /usr/lib/x86_64-linux-gnu/sasl2/libxoauth2.so
```

4 Verify the plugin is installed

```
# The symlink should point to the plugin library
ls -la /usr/lib/x86_64-linux-gnu/sasl2/libxoauth2.so

# You should also have the token helper tool
which sasl-xoauth2-tool
```

5. Install and Configure Postfix

1 Install Postfix

```
sudo apt install -y postfix libsasl2-modules

# During installation, select "Satellite system" or "Internet with
smarthost"
# You can reconfigure later with: sudo dpkg-reconfigure postfix
```

2 Configure `/etc/postfix/main.cf`

Edit the Postfix main configuration. Below are the key settings to add or modify:

```
# ── Relay Host ──────────────────────────────────────────
relayhost = [smtp.gmail.com]:587

# ── SASL Authentication ─────────────────────────────────
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
smtp_sasl_security_options =
smtp_sasl_mechanism_filter = xoauth2

# ── TLS Encryption ──────────────────────────────────────
smtp_tls_security_level = encrypt
smtp_tls_CAfile = /etc/ssl/certs/ca-certificates.crt
smtp_tls_wrappermode = no

# ── Sender Rewriting (optional) ─────────────────────────
# Force all outbound mail to come from the Gmail address
# Uncomment the next two lines if your legacy clients send
# with arbitrary From addresses:
# smtp_generic_maps = hash:/etc/postfix/generic
# sender_canonical_maps = hash:/etc/postfix/sender_canonical
```

> **Note:** Gmail will reject mail if the `From:` address doesn't match the authenticated account (or a configured alias). If your legacy clients send with a different `From:` address, you must configure sender rewriting. See **Section 8**.

3 Create the SASL password map

```
# Create the password file pointing to the token location
# The token path must be relative to the Postfix chroot
(/var/spool/postfix)
sudo bash -c 'cat > /etc/postfix/sasl_passwd << EOF
[smtp.gmail.com]:587    postfix-relay@gmail.com:/etc/tokens/postfix-
relay@gmail.com
EOF'

# Generate the hash database and lock down permissions
sudo postmap /etc/postfix/sasl_passwd
sudo chmod 600 /etc/postfix/sasl_passwd /etc/postfix/sasl_passwd.db
```

> **Important:** Replace `postfix-relay@gmail.com` with your actual Gmail or Workspace email address. This must match in **both** the sasl_passwd file and the token filename. The token path in `sasl_passwd` is chroot-relative (i.e., `/etc/tokens/...` not `/var/spool/postfix/etc/tokens/...`) because the Postfix `smtp` process runs inside the chroot.

6. Configure sasl-xoauth2

1 Create the sasl-xoauth2 configuration file

```
sudo bash -c 'cat > /usr/local/etc/sasl-xoauth2.conf << EOF
{
  "client_id": "<YOUR_CLIENT_ID>",
  "client_secret": "<YOUR_CLIENT_SECRET>",
  "token_endpoint": "https://oauth2.googleapis.com/token",
  "log_to_syslog_on_failure": "yes",
  "log_full_trace_on_failure": "no"
}
EOF'

sudo chmod 600 /usr/local/etc/sasl-xoauth2.conf
```

Replace `<YOUR_CLIENT_ID>` and `<YOUR_CLIENT_SECRET>` with the values from **Section 3, Step 4**.

> **Note:** The `sasl-xoauth2` plugin reads its configuration from `/usr/local/etc/sasl-xoauth2.conf` (the default install prefix). This file must also be copied into the Postfix chroot — see **Section 9**.

2 Create the token directory

```
# The token directory must be inside the Postfix chroot
sudo mkdir -p /var/spool/postfix/etc/tokens
sudo chown -R postfix:postfix /var/spool/postfix/etc/tokens
sudo chmod 700 /var/spool/postfix/etc/tokens
```

7. Obtain the OAuth2 Token

1 Generate the initial token

```
sudo sasl-xoauth2-tool get-token gmail \
  /var/spool/postfix/etc/tokens/postfix-relay@gmail.com \
  --client-id <YOUR_CLIENT_ID> \
  --client-secret <YOUR_CLIENT_SECRET> \
  --scope https://mail.google.com/
```

Replace `<YOUR_CLIENT_ID>` and `<YOUR_CLIENT_SECRET>` with the values from **Section 3, Step 4**.

This will:

1. Open a URL in your browser (or display one to visit manually)
2. Ask you to log in with the Gmail account
3. Ask you to authorize the application
4. Provide an authorization code to paste back into the terminal

> **Note:** If running on a headless server, copy the displayed URL to a browser on another machine. After authorizing, paste the authorization code back into the terminal session.

2 Verify the token file

```
# Check the token file was created and contains valid JSON
sudo cat /var/spool/postfix/etc/tokens/postfix-relay@gmail.com

# It should contain: access_token, refresh_token, expiry, etc.
```

3 Fix token file permissions

```
sudo chown postfix:postfix /var/spool/postfix/etc/tokens/postfix-
relay@gmail.com
sudo chmod 600 /var/spool/postfix/etc/tokens/postfix-relay@gmail.com
```

> **Token Refresh:** The `sasl-xoauth2` plugin automatically refreshes the access token using the stored refresh token. Access tokens typically expire after 1 hour, but the refresh token handles renewal transparently. However, if the refresh token itself is revoked (e.g., password change, consent revoked), you must re-run the `get-token` command.

8. Sender Rewriting (Optional)

Gmail enforces that the `From:` header matches the authenticated sender (or a configured alias in Gmail settings). If your legacy clients send as different addresses, set up sender rewriting:

Option A: Generic Table (rewrite all senders)

```
# /etc/postfix/generic
# Map any local sender to the Gmail address
@your-server.local    postfix-relay@gmail.com
```

```
# Add to /etc/postfix/main.cf:
smtp_generic_maps = hash:/etc/postfix/generic

# Generate the hash and reload
sudo postmap /etc/postfix/generic
sudo systemctl reload postfix
```

Option B: Sender Canonical Map (selective rewriting)

```
# /etc/postfix/sender_canonical
printer@local           postfix-relay@gmail.com
erp-system@local        postfix-relay@gmail.com
monitoring@local        postfix-relay@gmail.com
```

```
# Add to /etc/postfix/main.cf:
sender_canonical_maps = hash:/etc/postfix/sender_canonical

# Generate the hash and reload
sudo postmap /etc/postfix/sender_canonical
sudo systemctl reload postfix
```

9. Postfix Chroot Setup

Postfix runs in a chroot jail (`/var/spool/postfix`). Both the CA certificate bundle (for TLS to Gmail) and the `sasl-xoauth2` plugin configuration must be available inside the chroot.

```
# Copy the CA certificates into the chroot
sudo mkdir -p /var/spool/postfix/etc/ssl/certs
sudo cp /etc/ssl/certs/ca-certificates.crt
/var/spool/postfix/etc/ssl/certs/

# Copy the sasl-xoauth2 config into the chroot (matching the install
prefix path)
sudo mkdir -p /var/spool/postfix/usr/local/etc
sudo cp /usr/local/etc/sasl-xoauth2.conf
/var/spool/postfix/usr/local/etc/sasl-xoauth2.conf
sudo chmod 600 /var/spool/postfix/usr/local/etc/sasl-xoauth2.conf
```

> **Important:** Every time Postfix is restarted, files in the chroot may be cleared. To automate the copies, create a systemd override:

```
# Create the override directory
sudo mkdir -p /etc/systemd/system/postfix@-.service.d

# Create the override file
sudo bash -c 'cat >
/etc/systemd/system/postfix@-.service.d/override.conf << EOF
[Service]
ExecStartPre=/bin/cp /etc/ssl/certs/ca-certificates.crt
/var/spool/postfix/etc/ssl/certs/
ExecStartPre=/bin/mkdir -p /var/spool/postfix/usr/local/etc
ExecStartPre=/bin/cp /usr/local/etc/sasl-xoauth2.conf
/var/spool/postfix/usr/local/etc/sasl-xoauth2.conf
EOF'

# Reload systemd and restart Postfix
sudo systemctl daemon-reload
sudo systemctl restart postfix
```

10. Start Postfix and Test

1 Restart Postfix

```
sudo systemctl restart postfix
sudo systemctl enable postfix
```

2 Send a test email

```
# Using sendmail
echo "Subject: Postfix Gmail OAuth2 Test
From: postfix-relay@gmail.com
To: recipient@example.com

This is a test email sent through Postfix relaying via Gmail with
OAuth2." | sendmail -t

# Or using the mail command
echo "Test body" | mail -s "Postfix OAuth2 Test" -r postfix-
relay@gmail.com recipient@example.com
```

3 Monitor the mail log

```
# Watch the log in real-time
sudo journalctl -u postfix -f

# Or check the traditional mail log
sudo tail -f /var/log/mail.log
```

A successful delivery will show:

```
status=sent (250 2.0.0 OK ... - gsmtp)
```

11. Troubleshooting

Enable verbose logging for sasl-xoauth2

Edit `/usr/local/etc/sasl-xoauth2.conf` and set:

```
{
  ...
  "log_to_syslog_on_failure": "yes",
  "log_full_trace_on_failure": "yes"
}
```

Then copy the updated config into the chroot and restart Postfix:

```
sudo cp /usr/local/etc/sasl-xoauth2.conf
/var/spool/postfix/usr/local/etc/sasl-xoauth2.conf
sudo systemctl restart postfix
```

Check logs with `journalctl -u postfix -n 50`.

Common Issues

Symptom	Cause	Fix
`SASL authentication failed`	Token file missing, expired, or wrong permissions	Re-run `sasl-xoauth2-tool get-token` ; verify `chown postfix:postfix`
`relay access denied`	Postfix not configured as relay	Verify `relayhost` in `main.cf` ; check `mynetworks` includes your LAN
`certificate verification failed`	CA certs not in chroot	Copy `ca-certificates.crt` into chroot (see Section 9)
`550 5.7.0 Mail rejected`	`From:` address doesn't match Gmail account	Configure sender rewriting (Section 8) or add the address as a Gmail alias
`invalid_grant` in logs	Refresh token revoked or expired	Re-authorize: run `sasl-xoauth2-tool get-token gmail ...` again
`Token refresh failed`	Client ID/Secret mismatch or API disabled	Verify credentials in `/usr/local/etc/sasl-xoauth2.conf` ; ensure Gmail API is enabled in Cloud Console

Test token refresh manually

```
# Manually test that the token can be refreshed
sudo sasl-xoauth2-tool test-token-refresh \
  /var/spool/postfix/etc/tokens/postfix-relay@gmail.com
```

12. Gmail Sending Limits

> **Be aware of Gmail sending limits:**
>
> - **Free Gmail:** 500 emails per day (per rolling 24-hour period)
> - **Google Workspace:** 2,000 emails per day
> - Per-message recipient limit: 500 (free) / 2,000 (Workspace)
> - Exceeding limits results in temporary sending blocks (up to 24 hours)

For higher volume requirements with Google Workspace, consider using the **Google Workspace SMTP Relay** service instead (configured in the Admin console under Apps → Google Workspace → Gmail → Routing → SMTP relay service), which allows up to 10,000 messages per day.

13. Security Recommendations

- **Restrict relay access:** Configure `mynetworks` in `main.cf` to only allow trusted hosts:

  ```
  mynetworks = 127.0.0.0/8, 10.0.0.0/24
  ```

- **File permissions:** Ensure `/usr/local/etc/sasl-xoauth2.conf`, `sasl_passwd`, and token files are mode `600` and owned by `root` or `postfix`
- **Firewall:** Only expose Postfix (port 25) to your internal network; never to the public internet
- **Monitor logs:** Set up log monitoring for authentication failures
- **Token rotation:** Periodically review OAuth consent and connected apps in your Google account security settings

14. Quick Reference

Item	Value
SMTP Server	`smtp.gmail.com`
SMTP Port	`587` (STARTTLS)
SASL Mechanism	`XOAUTH2`
Token Endpoint	`https://oauth2.googleapis.com/token`
Authorization Endpoint	`https://accounts.google.com/o/oauth2/auth`
OAuth2 Scope	`https://mail.google.com/`
Token File Location	`/var/spool/postfix/etc/tokens/<email>`
Plugin Config	`/usr/local/etc/sasl-xoauth2.conf`
Postfix Config	`/etc/postfix/main.cf`
SASL Password Map	`/etc/postfix/sasl_passwd`

Appendix A: Complete main.cf Example

```
# /etc/postfix/main.cf - Gmail OAuth2 Relay Configuration

# ── General ──────────────────────────────────────────────
smtpd_banner = $myhostname ESMTP
biff = no
append_dot_mydomain = no
readme_directory = no

# ── Host Identity ────────────────────────────────────────
myhostname = relay.local
mydomain = local
myorigin = $myhostname
mydestination = $myhostname, localhost.$mydomain, localhost
mynetworks = 127.0.0.0/8, 10.0.0.0/24

# ── Relay ────────────────────────────────────────────────
relayhost = [smtp.gmail.com]:587
inet_interfaces = all
inet_protocols = ipv4

# ── SASL / OAuth2 ────────────────────────────────────────
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
smtp_sasl_security_options =
smtp_sasl_mechanism_filter = xoauth2

# ── TLS ──────────────────────────────────────────────────
smtp_tls_security_level = encrypt
smtp_tls_CAfile = /etc/ssl/certs/ca-certificates.crt
smtp_tls_wrappermode = no
smtp_tls_loglevel = 1

# ── Sender Rewriting (uncomment if needed) ───────────────
# smtp_generic_maps = hash:/etc/postfix/generic

# ── Limits ───────────────────────────────────────────────
mailbox_size_limit = 0
message_size_limit = 26214400
```

Appendix B: All Commands at a Glance

```
# 1. Install dependencies
sudo apt update
sudo apt install -y build-essential cmake pkg-config \
  libcurl4-openssl-dev libjsoncpp-dev libsasl2-dev postfix libsasl2-
modules

# 2. Build sasl-xoauth2
cd /tmp && git clone https://github.com/tarickb/sasl-xoauth2.git
cd sasl-xoauth2 && mkdir build && cd build && cmake .. && make
sudo make install
sudo ln -sf /usr/local/lib/sasl2/libsasl-xoauth2.so /usr/lib/x86_64-
linux-gnu/sasl2/libxoauth2.so

# 3. Create token directory
sudo mkdir -p /var/spool/postfix/etc/tokens
sudo chown -R postfix:postfix /var/spool/postfix/etc/tokens
sudo chmod 700 /var/spool/postfix/etc/tokens

# 4. Configure sasl-xoauth2.conf (edit with your Client ID & Secret)
sudo nano /usr/local/etc/sasl-xoauth2.conf

# 5. Configure Postfix main.cf
sudo nano /etc/postfix/main.cf

# 6. Create sasl_passwd and hash it
sudo nano /etc/postfix/sasl_passwd
sudo postmap /etc/postfix/sasl_passwd
sudo chmod 600 /etc/postfix/sasl_passwd /etc/postfix/sasl_passwd.db

# 7. Copy CA certs and sasl-xoauth2 config into chroot
sudo mkdir -p /var/spool/postfix/etc/ssl/certs
sudo cp /etc/ssl/certs/ca-certificates.crt
/var/spool/postfix/etc/ssl/certs/
sudo mkdir -p /var/spool/postfix/usr/local/etc
sudo cp /usr/local/etc/sasl-xoauth2.conf
/var/spool/postfix/usr/local/etc/sasl-xoauth2.conf

# 8. Obtain OAuth2 token (use your Client ID & Secret from step 4)
sudo sasl-xoauth2-tool get-token gmail \
  /var/spool/postfix/etc/tokens/postfix-relay@gmail.com \
  --client-id <YOUR_CLIENT_ID> \
  --client-secret <YOUR_CLIENT_SECRET> \
  --scope https://mail.google.com/
sudo chown postfix:postfix /var/spool/postfix/etc/tokens/postfix-
relay@gmail.com
```

```
sudo chmod 600 /var/spool/postfix/etc/tokens/postfix-relay@gmail.com

# 9. Start Postfix
sudo systemctl restart postfix
sudo systemctl enable postfix

# 10. Test
echo "Subject: Test" | sendmail -t recipient@example.com
sudo journalctl -u postfix -f
```